Sprint 16 · Reto «Entrega final: evaluar, demostrar y documentar»

Ficha completa del reto, para consultar. Para trabajar con ella abierta, usad **el reto en una página** (carpeta `1_Reto_en_una_pagina`, junto a esta ficha).

- Volver al README del sprint · README del sprint en PDF · ecofriendly
- **Esta ficha en PDF:** a color · ecofriendly, para imprimir
- Si es vuestro primer reto: Cómo se trabaja un reto.

RETO Ficha del reto

1 Qué os pide el cliente

Qué os pide el cliente. El cliente (el docente) va a **recibir vuestro trabajo como si fuera un entregable real**. Quiere poder **clonar el repositorio y seguir el README**, **ver funcionar el demostrador**, **leer una memoria de 10–15 páginas** y **entender los riesgos** antes de decidir si adopta vuestro modelo. Para eso tenéis que: evaluar a fondo el modelo candidato del Sprint 15 (una sola vez en el test), buscar sesgos por subgrupos y **aplicar y medir** una medida para reducirlos, cerrar el registro de riesgos y clasificar el sistema según el RGPD y el AI Act, construir un demostrador que no se rompa con entradas erróneas, que avise de la incertidumbre y que tenga **plan B en vídeo**, y documentarlo todo (model card, README y memoria técnica).

Cómo se trabaja. Este es el sprint de cierre: no hay una plantilla de código que rellenar (salvo el demostrador), sino **vuestro proyecto de los sprints 14 y 15**, que ahora hay que terminar y dejar listo para que otra persona lo use. Os damos plantillas para el demostrador, la model card, la memoria y la lista de congelación. **Todo lo que haya en la rama principal del repositorio a las 14:30 del 10/12 es lo que se evalúa**; pasada esa hora, se congela. Hay tres días de clase y, **entre el día 1 y el día 2, dos días sin clase**: buen momento para avanzar la memoria. El equipo decide hasta dónde llega: mínimo, estándar o extra.

2 · Recursos del reto

Recurso	Para qué lo usaréis	Dónde está
Vuestro proyecto (repositorio privado `ifcd107-equipo-…`)	El modelo candidato del Sprint 15, su registro de experimentos, los datos y el registro de riesgos de los sprints 14 y 15	Vuestro repositorio de GitHub
Cuaderno del S13 `S13_02`	Los gráficos de la evaluación (matriz de confusión, curvas, importancia de variables, SHAP o Grad-CAM) para adaptarlos	Colab
Presentación S16.1 y **apuntes del sprint**	Qué es un demostrador, cómo se documenta y, con ejemplos de números, cómo se miden los sesgos y su mitigación	Presentación · apuntes
Cuaderno del sprint `S16_01`	Práctica del demostrador: guardar el modelo y sus metadatos, separar la lógica, pruebas, Streamlit, widgets y API	Colab

Recurso	Para qué lo usaréis	Dónde está
La carpeta del demostrador `demostrador/`	`logica.py`, `app.py`, `api.py` (opcional), `tests/test_logica.py`, `requirements.txt` y `README_demostrador.md`. Es un ejemplo completo con el modelo de TurisData: vosotros lo adaptáis a **vuestro** proyecto. La carpeta `modelo/` no viene hecha: la crea el cuaderno `S16_01` o vuestro script de entrenamiento	En GitHub: *Code* → *Download ZIP*. Trabajad en una copia
Python y librerías (`scikit-learn`, `streamlit`, `pytest`…)	Ejecutar el cuaderno de evaluación, lanzar el demostrador y las pruebas	`pip install -r requirements.txt`. Si no podéis instalar Streamlit, la alternativa es el cuaderno con *widgets* de `S16_01`
Model card	Ficha técnica del modelo de máximo 2 páginas	`plantilla_model_card.md`
Esqueleto de la memoria	Secciones, presupuesto de páginas y mapa de requisitos R1–R11	`plantilla_esqueleto_memoria_tecnica.md`
Lista de congelación	Repasar todo antes de las 14:30	`checklist_congelacion.md`
Coevaluación individual (anexo B)	Valorar el trabajo de cada persona del equipo. Es **confidencial**: va al docente, no al repositorio	`plantilla_coevaluacion.md`
Cómo entregar en GitHub	Subir los ficheros a `sprint-16/`	Guía de entrega
Autoevaluación y coevaluación	Cada persona rellena su hoja y la sube	Hoja del sprint
Un asistente de IA (opcional)	Entender sesgos, errores de código o revisar la redacción	Con las reglas de la «Ayuda 1» del paso a paso

3 · Qué entregar

Este apartado responde a **dos preguntas distintas**, una tabla para cada una:

Tabla A · Los ficheros: qué ficheros hay, de dónde sale cada uno (ya hecho, lo modificáis, lo creáis o lo genera el programa) y **cuáles se suben y adónde**.

Tabla B · Los requisitos: qué tiene que cumplir lo que entregáis y hasta dónde llegar (mínimo, estándar, extra). **Es lo que se evalúa.**

En total, al final del sprint se suben a `sprint-16/`: 9 ficheros del equipo (`README.md`, `notebooks/04_evaluacion.ipynb`, `demostrador/logica.py`, `demostrador/app.py`, `demostrador/tests/test_logica.py`, `demostrador/requirements.txt`, `docs/riesgos_eticos.md`, `docs/model_card.md`, `docs/memoria_tecnica.pdf`) **+ 1 autoevaluación y coevaluación por persona**, es decir, **9 + tantos ficheros como personas haya en el equipo** (un equipo de 4 sube 13). «El reto» no es un fichero aparte: es ese conjunto, y las carpetas `notebooks/`, `demostrador/` y `docs/` van **dentro de `sprint-16/`**. Además: el vídeo plan B va a `docs/` solo si pesa menos de 25 MB (si no, su enlace va en el README); el modelo con su `metadatos.json` y los datos solo si pesan poco (si no, el README explica cómo generarlos o descargarlos); y la coevaluación confidencial (anexo B) **no se sube al repositorio**: se entrega al docente. Lo que ya subisteis en los sprints 14 y 15 (datos, código, registro de experimentos) no hay que subirlo otra vez. La *review* y la *retro* se hacen de palabra: no se suben.

Tabla A · Los ficheros: de dónde sale cada uno y cuáles se suben

Todo lo que se sube va al repositorio privado del equipo (`ifcd107-equipo-…`), en la carpeta **`sprint-16/`**, salvo los datos.

Fichero	Origen	Qué hacéis vosotros	¿Se sube?
`README.md`	**Lo creáis.** No hay plantilla: la lista de apartados está en el bloque A de la lista de congelación	Escribirlo: problema, resultados, estructura de carpetas, cómo reproducir, cómo lanzar la demo, datos y licencias, IA responsable y equipo. Al final, la sección «Revisión con la rúbrica»	**Sí** · `sprint-16/`
`notebooks/04_evaluacion.ipynb`	**Lo creáis a partir de un cuaderno.** Os damos `S13_02` con los gráficos	Adaptarlo a vuestro modelo: evaluación en test, gráficos, análisis de errores, rendimiento por subgrupos y mitigación medida	**Sí** · `sprint-16/notebooks/`
`demostrador/logica.py`	**Lo modificáis.** Plantilla con validación, nivel de riesgo y aviso de incertidumbre	Cambiar los campos, las reglas de validación y el texto del aviso por los de vuestro problema	**Sí** · `sprint-16/demostrador/`

Fichero	Origen	Qué hacéis vosotros	¿Se sube?
`demostrador/app.py`	**Lo modificáis.** Plantilla de interfaz con Streamlit	Adaptar el formulario a vuestros campos. Si usáis Gradio u otra herramienta, el fichero se llama distinto pero cuenta igual	**Sí** · `sprint-16/demostrador/`
`demostrador/tests/test_logica.py`	**Lo modificáis.** Pruebas de sentido común para `pytest`	Cambiar la reserva de ejemplo por una de vuestro proyecto y añadir tres pruebas propias	**Sí** · `sprint-16/demostrador/tests/`
`demostrador/requirements.txt`	**Lo modificáis.** Trae las versiones del entorno de la plantilla	Recortarlo a lo que usáis y añadir lo que use el cuaderno de evaluación. Indicad la versión de Python en el README	**Sí** · `sprint-16/demostrador/`
`docs/riesgos_eticos.md`	**Lo modificáis.** Es el registro de riesgos que empezasteis en el Sprint 14 y revisasteis en el 15	Cerrarlo: cada riesgo alto con su estado y su evidencia, y la clasificación RGPD / AI Act justificada	**Sí** · `sprint-16/docs/`
`docs/model_card.md`	**Lo creáis a partir de una plantilla.** `plantilla_model_card.md`	Rellenar sus 9 apartados con las cifras finales. Máximo 2 páginas	**Sí** · `sprint-16/docs/`

Fichero	Origen	Qué hacéis vosotros	¿Se sube?
`docs/memoria_tecnica.pdf`	**Lo creáis a partir de una plantilla.** `plantilla_esqueleto_memoria_tecnica.md`	Escribir el texto en el programa que uséis y exportarlo a PDF. 10–15 páginas sin contar anexos	**Sí** · `sprint-16/docs/`
`docs/demo.mp4` (vídeo plan B)	**Lo creáis.** Grabación de pantalla	Grabar 1–2 minutos del demostrador funcionando	Solo si pesa menos de 25 MB · `sprint-16/docs/` ; si no, su enlace en el README
`demostrador/modelo/` (modelo y `metadatos.json`)	**Lo genera** el cuaderno `S16_01` o vuestro script de entrenamiento	Ejecutarlo; no se escribe a mano	Solo si pesan poco; si no, el README explica cómo generarlos · `sprint-16/demostrador/`
`demostrador/api.py` (extra)	**Lo modificáis.** Plantilla de API con FastAPI	Solo si el equipo llega al nivel extra	Solo si lo hacéis · `sprint-16/demostrador/`
Datos del proyecto o script de descarga	**Ya hecho** en el Sprint 14	Nada: comprobad que el README dice de dónde salen y con qué licencia	Solo si faltan y pesan poco · `data/`
`checklist_congelacion.md`	**Ya hecha, en blanco**	Pasarla entera con una persona que no escribió el README	**No** se sube: se rellena en papel o en el tablero
Coevaluación individual (anexo B)	**Ya hecha, en blanco**	Cada persona rellena la suya	**No** se sube al repositorio: se entrega al docente
Autoevaluación y coevaluación (`autoevaluacion_nombre.md`)	**Ya hecha, en blanco**	Cada persona copia la plantilla con su nombre y la rellena; la coevaluación de la *review* va **dentro del mismo fichero**	**Sí, una por persona** (no una por equipo) · `sprint-16/`

Tabla B · Los requisitos: qué tiene que cumplir y hasta dónde llegar

Requisito	Qué se pide exactamente	Nivel	Dónde se comprueba
Evaluación final en el test	Se evalúa **una sola vez** el modelo candidato del Sprint 15, con las métricas y su incertidumbre, y gráficos adaptados de `S13_02` (matriz de confusión, curvas, importancia de variables), cada uno con un título que dice la conclusión	Mínimo	`notebooks/04_evaluacion.ipynb`
Rendimiento por subgrupos	La misma métrica para cada subgrupo relevante (canal, país, tipo de imagen…). Si hay diferencias grandes, se anota el sesgo	Mínimo	Cuaderno y apartado 6 de la model card
Análisis de errores	Lista de los casos donde más falla, con ejemplos reales y lo que tienen en común	Estándar	Cuaderno y memoria (sección 5)
Mitigación del sesgo	Una medida elegida y aplicada: reponderar, ajustar el umbral por grupo, añadir datos del grupo que falta o limitar el uso	Mínimo	Cuaderno y memoria (sección 6)
Mitigación medida	Métricas **antes y después, por subgrupo**: qué mejora, qué empeora y qué cuesta (por ejemplo, más falsas alarmas). Si no se puede arreglar, se documenta como limitación	Estándar	Cuaderno, model card y memoria (sección 6)
Riesgos y normativa	Registro cerrado: cada riesgo alto con estado y evidencia. Clasificación RGPD / AI Act justificada, con supervisión humana y transparencia	Mínimo	`docs/riesgos_eticos.md` y model card
Demostrador	Se lanza con **un comando** indicado en el README y funciona desde cero en otro equipo. No guarda datos personales ni lleva claves	Mínimo	`demostrador/` y `README.md`
Entradas erróneas e incertidumbre	Se prueba con 3 entradas normales, 1 imposible y 1 vacía: mensajes claros y sin romperse. Cada respuesta lleva un aviso de incertidumbre (por ejemplo, «confianza: media»)	Estándar	`demostrador/logica.py` y `app.py`
Pruebas automáticas	`pytest -q` pasa, con al menos tres pruebas propias de sentido común	Estándar	`demostrador/tests/test_logica.py`
Plan B en vídeo	Vídeo de 1–2 minutos (unos 90 segundos) del demostrador funcionando	Estándar	`docs/demo.mp4` o enlace en el README
README y versiones	README con los apartados indicados y `requirements.txt` con **versiones fijas** y versión de Python	Mínimo	`README.md` y `demostrador/requirements.txt`
Prueba en limpio	Una persona que **no escribió el README** lo sigue en un Colab nuevo o una carpeta limpia (*Code → Download ZIP*) y lo anota. Se corrige lo que falle	Estándar	`README.md` (sección «Revisión con la rúbrica»)
Model card	Las 9 partes de la plantilla, máximo 2 páginas. Coincide con la memoria y con el aviso del demostrador	Mínimo	`docs/model_card.md`
Memoria técnica	PDF de 10–15 páginas sin contar anexos, propia y autocontenida, con las secciones del esqueleto, figuras con pie y referencias. Si usáis IA para redactar, se revisa y se declara	Mínimo	`docs/memoria_tecnica.pdf`

Requisito	Qué se pide exactamente	Nivel	Dónde se comprueba
Cifras coherentes y conclusiones accionables	Las mismas cifras en memoria, README, model card y registro de experimentos. La memoria dice qué recomendáis al cliente y qué **no** debe hacerse con el modelo	Estándar	`docs/memoria_tecnica.pdf` (sección 9)
Git y congelación	Commits de **todas** las personas. Etiqueta `v1.0-entrega` creada **antes de las 14:30 del 10/12**. Ni claves ni datos personales. Nada se cambia después	Mínimo	Historial y etiquetas del repositorio
Lista de congelación	Los bloques A a F pasados enteros, con una persona que no escribió el README	Estándar	`checklist_congelacion.md` (en papel o en el tablero)
Coevaluación individual (anexo B)	Cada integrante entrega la suya al docente. Junto con los commits, ajusta la nota individual (de 0,8 a 1,1)	Mínimo	Entrega al docente
Revisión y autoevaluación	Alguien que **no escribió esa parte** revisa el repositorio y la memoria con la rúbrica (justo debajo de estas tablas) y anota nivel y prueba en el README; cada persona sube su hoja de autoevaluación y coevaluación	Estándar	`README.md` (sección «Revisión con la rúbrica») y `sprint-16/`
Extras	SHAP o Grad-CAM interpretado con palabras · métricas con intervalo de incertidumbre · comparar dos medidas de mitigación · comprobar el rendimiento con datos distintos (otro periodo, fotos reales…) · `api.py` con `/salud` y `/predecir`	Extra	Cuaderno · memoria · `demostrador/api.py`

Rúbrica del reto · cómo se puntúa y cómo se revisa

La usan **dos veces**: el equipo, para revisar su propio trabajo antes de congelar (es lo que dice la *Definition of Done*), y el docente, para puntuar. Se mira **cada criterio por separado** y se pone el nivel de la columna cuyo texto **se cumple entero**: si lo de un nivel se cumple a medias, es el nivel de abajo. **0** significa que no se ha entregado nada de ese criterio.

Orientación: lo que pide el nivel mínimo sale en un **2**, el estándar en un **3** y los extras en un **4**. Esta rúbrica sirve para la nota del sprint; la nota del **proyecto** usa la rúbrica de 100 puntos del documento de briefs.

Criterio	1 · Empezado	2 · Mínimo	3 · Estándar	4 · Extra
Corrección técnica (¿funciona?)	Hay evaluación, pero el test se ha usado más de una vez o solo hay una métrica global; el demostrador no arranca	Evaluación en test hecha una vez con métricas y gráficos; métrica por subgrupos; sesgo anotado y medida elegida y aplicada; el demostrador arranca en el equipo de quien lo hizo	Además, análisis de errores con ejemplos reales; mitigación medida antes y después por subgrupo; el demostrador rechaza entradas erróneas con un mensaje claro y `pytest -q` pasa	Además, el modelo se interpreta con SHAP o Grad-CAM, hay intervalos de incertidumbre, se comparan dos mitigaciones o se prueba con datos distintos
Reproducibilidad (¿lo puede repetir otra persona?)	Solo funciona en el ordenador de quien lo hizo; el README no dice cómo ejecutarlo	El README tiene los comandos y hay `requirements.txt`, pero faltan versiones, datos o la versión de Python	Otra persona, en un Colab nuevo o una carpeta limpia, lo ejecuta **siguiendo solo el README** y ve el demostrador; versiones fijas; commits de todas las personas; etiqueta `v1.0-entrega` creada antes de las 14:30	Además, el README indica el tiempo total de reproducción, la estructura de carpetas y un comando único para la demo, y la prueba en limpio está anotada (quién, qué falló, cómo se corrigió)

Criterio	1 · Empezado	2 · Mínimo	3 · Estándar	4 · Extra
Análisis y comunicación al cliente (¿lo entiende quien no programa?)	La memoria está incompleta o la model card tiene huecos esenciales	Memoria de 10–15 páginas con todas las secciones y model card completa, pero con conclusiones genéricas o cifras que no coinciden	Memoria clara y autocontenida, con figuras con pie y referencias; cifras iguales en todos los documentos; conclusiones accionables (qué hacer y qué **no** hacer con el modelo); el demostrador comunica la incertidumbre	Además, un resumen ejecutivo que se entiende sin leer el resto y recomendaciones priorizadas con lo que habría que hacer antes de un despliegue real
IA responsable (¿se sabe qué riesgos hay y qué se hizo?)	El registro de riesgos no está cerrado o no hay análisis por subgrupos	Registro con estado de cada riesgo alto, clasificación RGPD / AI Act citada, sesgo detectado y aviso de límites en el demostrador	Clasificación **justificada**; mitigación aplicada **y medida**; limitaciones y usos no permitidos en la model card; el uso de IA declarado (qué se pidió, qué se revisó y qué se cambió); sin datos personales ni claves	Además, supervisión humana y transparencia explicadas, y se cuenta un fallo concreto (de la IA o de la mitigación, por ejemplo, un subgrupo que empeoró) y qué se decidió

Cómo se revisa (unos 20 minutos, antes de crear la etiqueta; lo hace **una persona que no escribió esa parte**):

1. **Ejecutar**, en una carpeta limpia o en un Colab nuevo, lo que dice el README: instalar, abrir el cuaderno de evaluación, lanzar el demostrador y ejecutar `pytest -q`, copiando solo lo que pone ahí. Anotar lo que falla y cuánto se tarda.
2. **Probar el demostrador** con 3 entradas normales, 1 imposible y 1 vacía: ¿mensajes claros? ¿avisa de la incertidumbre?
3. **Leer la memoria y la model card como si fuerais el cliente**: ¿se entiende qué recomendáis y cuándo no fiarse del modelo? Comprobar que las cifras son iguales en todos los documentos y que el registro de riesgos está cerrado.
4. **Poner un nivel (1 a 4) en cada criterio, con una frase de prueba** (por ejemplo: «3 · el README funciona en un Colab nuevo, pero falta la versión de Python»). Sin prueba escrita, el nivel no vale.
5. **Apuntar el resultado** en la sección «Revisión con la rúbrica» del `README.md` y corregir lo más fácil de arreglar. Lo demás se anota como mejora.

Cada persona, además, se puntúa a sí misma en su hoja de autoevaluación y coevaluación con **esta misma rúbrica**.

---

4 · Guía paso a paso

Son ocho pasos, de **525 minutos en total** repartidos en tres días (en el calendario hay unos 470 minutos de reto en clase). Cada uno dice **qué hacéis** (la lista corta), **cómo se hace** con el detalle, **qué deberíais ver** y **cuándo está hecho**. Las ayudas (IA, errores, atascos) están al final. El día 1 empieza con 10 minutos de planning, la píldora de teoría y la práctica del demostrador (`S16_01`), que no se cuentan en los 525. Si el tiempo aprieta, completad el nivel mínimo y dejad lo demás para los dos días sin clase.

Paso 1 · Evaluar el modelo en el test y buscar dónde falla (día 1, 40 minutos)

1. Evaluar el modelo candidato del Sprint 15 en el test, **una sola vez**, y anotar las métricas con su incertidumbre.
2. Adaptar los gráficos de `S13_02` y guardar todo en `notebooks/04_evaluacion.ipynb`.
3. Calcular la métrica por subgrupos y sacar la lista de los errores más frecuentes.

Cómo se hace

- **Evaluar.** Cargad el modelo ya elegido y pasadle el test. **Una vez.** Si el resultado no os gusta, no se vuelve a elegir mirando el test: se anota y se explica. Guardad la métrica con su intervalo o su variación.

- **Gráficos.** Abrid `S13_02` y copiad lo que os sirva: matriz de confusión, curvas, importancia de variables, SHAP o Grad-CAM. Cada gráfico con un **título que diga la conclusión** («Las reservas por móvil se cancelan más»), no una descripción.
- **Subgrupos.** Elegid las columnas con categorías (país, canal, tipo de imagen…) y calculad la métrica para cada valor. Ejemplo con cifras inventadas: el *recall* es del 74,8 % en total, pero del 80 % en web y del 58 % en móvil. Eso es un sesgo que el número global esconde. Si el subgrupo tiene pocos casos, decid cuánta incertidumbre hay.
- **Errores.** Sacad la lista de los casos donde más falla (por ejemplo, las 20 cancelaciones que el modelo no detectó), mirad ejemplos reales uno a uno y anotad qué tienen en común. De ahí salen las limitaciones honestas de la memoria.
- **Tutoría.** En estos 40 minutos cada equipo tiene su franja de tutoría de sesgos y visualización: llevad el cuaderno abierto con la métrica por subgrupos.

Qué deberíais ver. Una tabla con las métricas del test y su incertidumbre, dos o tres gráficos con título-conclusión, una tabla con la métrica de cada subgrupo (aunque no haya diferencias, se anota) y una lista corta de errores típicos.

Hecho cuando: el cuaderno `04_evaluacion.ipynb` tiene la evaluación en test, los gráficos, los subgrupos y la lista de errores, y todo el equipo sabe qué subgrupo va peor.

Paso 2 · Abrir la memoria y la model card (día 1, 85 minutos)

1. Repartir las secciones de la memoria entre las personas del equipo.
2. Copiar el esqueleto de la memoria y la plantilla de la model card y rellenar lo que ya sabéis.
3. Dejar el demostrador básico funcionando y anotar lo que falta y quién lo hace.

Cómo se hace

- **Repartir.** Abrid `plantilla_esqueleto_memoria_tecnica.md`: tiene las secciones 0 a 9, un presupuesto de páginas (total 10–15) y un mapa de requisitos R1–R11. Cada persona redacta su parte, con **vuestros** resultados. El resumen ejecutivo (sección 0) se escribe **al final**.
- **Rellenar lo que ya existe.** Las secciones 1 a 4 salen de lo que hicisteis en los sprints 14 y 15 (ficha de proyecto, datos y EDA, comparativa de modelos). Se trata de contar, no de inventar.
- **Model card.** Copiad `plantilla_model_card.md` como `docs/model_card.md` y rellenad los apartados 1 a 5. Los demás se cierran el día 2.
- **Demostrador.** Partid de lo que hicisteis en la práctica `S16_01` y de la carpeta `demostrador/`. Basta con que arranque con vuestro modelo.
- **Los dos días sin clase.** Dejad apuntado en el tablero qué avanza cada persona para el día 2. El borrador de las secciones 1 a 6 debería estar listo ese día.

Qué deberíais ver. Un documento de memoria con todas las secciones del esqueleto (aunque algunas estén a medias), una model card con los primeros apartados y un demostrador que arranca.

Hecho cuando: existe el esqueleto de la memoria con una persona responsable por sección, la model card está empezada y el demostrador básico funciona.

Paso 3 · Mitigar el sesgo y volver a medir (día 2, 80 minutos)

1. Elegir una medida de mitigación para el sesgo que detectasteis.
2. Aplicarla y volver a medir, por subgrupo, antes y después.
3. Anotar los resultados en el cuaderno y en el registro de experimentos.

Cómo se hace

- **Elegir (5 minutos).** Una de estas, la que mejor encaje con vuestro sesgo:

Medida	En qué consiste
Reponderar	Dar más peso al entrenar a los ejemplos del subgrupo que va peor
Ajustar el umbral por grupo	Decidir «cancelará» con una probabilidad mínima distinta para cada grupo

Medida	En qué consiste
Añadir datos	Conseguir más ejemplos del subgrupo que falta
Limitar el uso	Decir en el aviso del demostrador: «no usar el modelo para el grupo X»

- **Aplicar y medir (60 minutos).** Repetid la medición del paso 1 con la mitigación puesta y comparad **antes y después**, subgrupo a subgrupo. Mirad también el precio que se paga: si el *recall* sube, quizá la precisión baje y haya más falsas alarmas. No hay mitigación gratis; la memoria explica ese equilibrio.
- **Si el sesgo no se puede arreglar.** Faltan datos y no hay forma de conseguirlos: entonces se documenta como limitación, se avisa en el demostrador y se limita el uso. Esconderlo no es una opción.
- **Registro (15 minutos).** Apuntad la medida, las cifras de antes y de después y la decisión en el registro de experimentos del proyecto, y copiad la tabla al cuaderno.

Qué deberíais ver. Una tabla con el antes y el después por subgrupo y una frase de lectura honesta, del estilo: «El *recall* de móvil sube del 58 % al 68 % y el de web baja un punto; a cambio hay más falsas alarmas» (cifras de ejemplo).

Hecho cuando: hay una mitigación aplicada, medida por subgrupo y anotada en el cuaderno y en el registro de experimentos.

Paso 4 · Pulir el demostrador y probarlo en limpio (día 2, 40 minutos)

1. Adaptar `logica.py` a los campos, las validaciones y el aviso de incertidumbre de vuestro proyecto.
2. Probar entradas normales, imposibles y vacías.
3. Escribir las pruebas con `pytest` y fijar las versiones en `requirements.txt`.
4. Lanzarlo en una carpeta nueva siguiendo solo el README.

Cómo se hace

- **Adaptar.** En `logica.py` cambiad los campos, los rangos válidos y el texto del aviso (cuándo no fiarse, qué no se debe hacer). Los errores, en español y diciendo qué campo falla, qué se esperaba y qué se recibió. La respuesta **devuelve datos**, no imprime: la interfaz es quien los muestra. Si vuestro demostrador es un asistente o *chatbot*, tiene que decir que es una IA y citar sus fuentes.
- **Probar a propósito.** 3 entradas normales, 1 imposible (una edad de 250) y 1 vacía. Debe salir un mensaje claro, por ejemplo «Introduce una edad entre 0 y 120», y nunca un texto rojo de error. Al predecir, debe mostrar algo como «Riesgo: alto (confianza media)».
- **Pruebas.** En `tests/test_logica.py` cambiad la reserva de ejemplo por una de vuestro proyecto y añadid tres pruebas propias de sentido común (por ejemplo: «una reserva flexible y sin depósito tiene más riesgo que una no reembolsable con depósito total»). Desde la carpeta `demostrador/`: `pytest -q`.
- **Versiones.** Dejad en `requirements.txt` solo lo que usáis, con versión fija, y anotad la versión de Python en el README.
- **Lanzar.** `pip install -r requirements.txt` y `streamlit run app.py` (se abre en `http://localhost:8501`). La API es opcional.

Qué deberíais ver. La app abierta en el navegador, mensajes claros con las entradas malas, un aviso de incertidumbre en cada resultado y `pytest -q` sin fallos.

Hecho cuando: el demostrador se lanza con un comando, no se rompe con las cinco entradas de prueba y las pruebas pasan.

Paso 5 · Cerrar la model card y completar la memoria (día 2, 80 minutos)

1. Terminar la model card con las cifras finales (35 minutos).
2. Clasificar el sistema según el RGPD y el AI Act y actualizar el registro de riesgos.
3. Completar el borrador de la memoria con todos los apartados (45 minutos).

Cómo se hace

- **Model card.** Rellenad los apartados que faltan: subgrupos (con las cifras del paso 3), ética y riesgos, limitaciones («cuándo no fiarse») y mantenimiento. Debe coincidir con la memoria y con el aviso del demostrador. Máximo 2 páginas.
- **Riesgos y normativa.** Decidid si el sistema es de riesgo mínimo, limitado o alto y si hay datos personales, y justificadlo. Cada riesgo alto del registro `docs/riesgos_eticos.md` necesita un **estado** (mitigado, aceptado o pendiente) y una **evidencia** (dónde se ve en el repositorio). «Hemos cuidado la privacidad» no vale; «no hay datos personales, ver `docs/plan_datos.md`» sí.
- **Memoria.** Completad todas las secciones del esqueleto, aunque sea en borrador, con las figuras del cuaderno (con pie y numeradas). Respetad el presupuesto de páginas. Escribid con vuestras palabras: si usáis IA para redactar, revisadlo y anotadlo para declararlo.

Qué deberíais ver. Una model card sin huecos, un registro de riesgos con estado y evidencia en cada riesgo alto y una memoria con todas las secciones escritas.

Hecho cuando: la model card está completa, la clasificación RGPD / AI Act está justificada y la memoria tiene todos los apartados.

Paso 6 · Verificar en limpio y corregir (día 3, 80 minutos)

1. Una persona que no escribió el README lo sigue en un Colab nuevo o una carpeta limpia (60 minutos).
2. Mientras tanto, el resto cierra la memoria y la exporta a PDF.
3. Corregir lo que haya fallado (20 minutos).

Cómo se hace

- **La prueba en limpio.** Descargad el repositorio como ZIP (*Code* → *Download ZIP*), abrid el cuaderno principal en Colab y seguid **solo** el README. Si lo prueba quien lo escribió, no vale: ya sabe los trucos que no están anotados. Apuntad cada fallo con su mensaje.
- **La memoria.** Pasad a limpio el texto, comprobad que tiene entre 10 y 15 páginas sin contar portada, índice, referencias ni anexos, y exportadla como `docs/memoria_tecnica.pdf`. Repasad la lista de calidad del final del esqueleto: cada figura con título-conclusión y citada en el texto, cifras iguales que en el README, y revisada por alguien que no la escribió.
- **Corregir.** Lo más habitual: dependencias sin documentar (añadidlas a `requirements.txt`), rutas que solo existen en un ordenador y datos que faltan. Después, repetid la parte que falló.

Qué deberíais ver. Una lista de fallos pequeña y ya resuelta, el demostrador funcionando en el entorno limpio y la memoria en PDF.

Hecho cuando: otra persona ha reproducido el proyecto con solo el README, lo que falló está corregido y existe `docs/memoria_tecnica.pdf`.

Paso 7 · Plan B, riesgos cerrados, lista de congelación y coevaluación (día 3, 65 minutos)

1. Grabar el vídeo plan B del demostrador (30 minutos).
2. Cerrar el registro de riesgos y pasar la lista de congelación (35 minutos).
3. Rellenar vuestra coevaluación individual (anexo B).

Cómo se hace

- **Vídeo plan B.** Grabad la pantalla (1–2 minutos, unos 90 segundos) con el demostrador funcionando y una entrada errónea a propósito. Guardadlo como `docs/demo.mp4` si pesa menos de 25 MB; si no, subidlo a un sitio estable y poned el enlace en el README.
- **Lista de congelación.** Abrid `checklist_congelacion.md` y marcad los bloques A (repositorio), B (reproducibilidad), C (demostrador), D (evaluación e IA responsable), E (memoria) y F (equipo y metodología). Debe hacerlo **una persona distinta a la que escribió el README**. Cada casilla sin marcar es un riesgo.
- **Sin claves ni datos personales.** Revisad que no hay ninguno en ningún fichero. Si encontráis uno, avisad al docente.
- **Coevaluación (anexo B).** Cada persona rellena `plantilla_coevaluacion.md`: puntúa de 1 a 4 a cada integrante (incluida ella) y estima el reparto del trabajo. Un 1 o un 4 llevan un ejemplo. Se entrega al docente, no al repositorio.

Qué deberíais ver. El vídeo guardado, la lista de congelación con todas las casillas marcadas (o con lo pendiente apuntado) y cada persona con su coevaluación hecha.

Hecho cuando: el vídeo plan B existe, el registro de riesgos está cerrado, la lista de congelación está pasada y todo el equipo ha entregado la coevaluación.

Paso 8 · Revisión con la rúbrica, review, retro y congelación (día 3, 55 minutos)

1. Revisar el trabajo con la rúbrica y anotar el resultado en el README.
2. Hacer la review (unos 5 minutos por equipo) y la retro.
3. Subir todo a `sprint-16/`, crear la etiqueta `v1.0-entrega` y subir las autoevaluaciones, **antes de las 14:30**.

Cómo se hace

- **La revisión con la rúbrica (20 minutos, antes de la etiqueta).** Quien revisa sigue los cinco pasos de «Cómo se revisa» (apartado 3) y rellena al final del `README.md` la sección **«Revisión con la rúbrica»**: un nivel de 1 a 4 por criterio y **una frase de prueba**. Lo más fácil de arreglar se arregla ya; lo demás queda como «lo primero que mejoraríamos».
- **La review.** Unos 5 minutos por equipo, de los que 4 son de demostración del demostrador, con una entrada errónea a propósito. Enseñad también una diapositiva con el análisis de sesgos y la medida aplicada, y la lista de riesgos altos con su estado. El docente propone **una entrada difícil** para probar el demostrador en directo. En la **retro**, una cosa que mantener y una que cambiar, con una acción concreta.
- **Subir los ficheros.** Seguid Cómo entregar en GitHub: (1) abrid el repositorio del equipo; (2) *Add file* → *Create new file* y escribid la ruta completa, por ejemplo `sprint-16/docs/model_card.md` (la barra crea la carpeta), o *Upload files* arrastrando la carpeta; (3) mensaje corto y **Commit changes**. Los ficheros, **en minúsculas y sin espacios**. Que suban personas distintas: el historial muestra a todo el equipo.
- **La etiqueta.** En la página principal del repositorio, a la derecha, *Releases* → *Create a new release*; en *Choose a tag* escribid `v1.0-entrega` y elegid *Create new tag*; el destino es `main`; poned un título (por ejemplo «Entrega final») y pulsad *Publish release*. Los nombres de los botones pueden cambiar un poco. Con Git en el terminal: `git tag v1.0-entrega` y `git push origin v1.0-entrega`. Comprobad que la etiqueta aparece. **Después de las 14:30 no se cambia código ni memoria.** Un fallo crítico del demostrador solo se corrige con permiso del docente y dejándolo documentado.
- **Las autoevaluaciones.** Cada persona sube su `autoevaluacion_nombre.md` a `sprint-16/` con su propia cuenta.

Qué deberíais ver. En `sprint-16/` los 9 ficheros del equipo y una hoja de autoevaluación por persona (**9 + N**); en el historial, commits de todas las personas; y la etiqueta `v1.0-entrega` con una hora anterior a las 14:30.

Hecho cuando: la sección «Revisión con la rúbrica» del README está rellena, todo está subido, la etiqueta existe antes de las 14:30 y cada persona ha subido su autoevaluación y coevaluación.

Ayuda 1 · Cómo usar la IA en este reto

La IA puede ahorrar tiempo, pero **escribe con seguridad cosas que pueden ser falsas** y puede cambiar una cifra sin avisar. Reglas:

- **Todo lo que dé, lo ejecutáis y lo entendéis.** Si el docente pregunta por una parte, tenéis que saber explicarla.
- **La memoria es vuestra:** la IA no la escribe por vosotros. Si la usáis para redactar o corregir, revisadla entera y **declaradlo en la memoria**.
- **Las cifras mandan:** después de que la IA toque un texto, comprobad que no ha cambiado ninguna.
- **Nada de datos personales ni claves** en una IA.

Para qué	Ejemplo de petición	Qué comprobáis
Entender sesgos	«Explícame con un ejemplo qué significa que un modelo tenga peor rendimiento en un subgrupo y qué medidas existen»	Que podéis aplicar una medida **y medirla**
Crear la app	«Ayúdame a añadir validación de entradas a esta app de Streamlit: *[código]*»	Que **probáis** entradas absurdas

Para qué	Ejemplo de petición	Qué comprobáis
Estructurar la memoria	«Dame un guion de las secciones de una memoria técnica de un proyecto de IA»	Que cada sección la redactáis vosotros con **vuestros resultados**
Revisar la redacción	«Corrige la ortografía y la claridad de este párrafo sin cambiar el contenido»	Que no se ha cambiado ninguna cifra

Ayuda 2 · Errores frecuentes (y cómo evitarlos)

Error	Por qué ocurre	Cómo evitarlo
Dejar la memoria para el último día	«Total, ya está hecho»	Borrador el día 2 (9/12) y revisión el día 3 (10/12)
El demostrador solo funciona en el ordenador de una persona	Dependencias sin documentar	`requirements.txt` con versiones y prueba en un entorno limpio
Mitigación que se aplica pero no se mide	Se piensa que basta con hacerla	Comparad las métricas **antes y después, por subgrupo**
Subir cambios después de las 14:30	Se siguen retocando cosas	Crear la etiqueta antes y no tocar nada después
Coevaluación sin entregar	Se olvida	Cada persona la entrega; cuenta para la nota individual
Memoria con más de 15 páginas o con fragmentos copiados	Se llena de relleno	Respetad el presupuesto de páginas; los anexos no cuentan; escribid con vuestras palabras
Cifras distintas en la memoria y en el README	Se copian de versiones distintas	Una revisión final de coherencia antes de congelar
Solo una persona hace commits	«El resto no sabe Git»	Reparto real: vale que cada persona suba un fichero desde la web

Ayuda 3 · Si os atascáis

1. **El demostrador no arranca en otro ordenador:** faltan dependencias. Generad `requirements.txt` con `pip freeze` (o a mano), recortadlo a lo que usáis y probad en un entorno nuevo.
2. **No sabéis qué sesgo medir:** mirad las columnas con categorías (país, canal, tipo) y calculad la métrica para cada valor.
3. **Falta tiempo:** priorizad lo obligatorio (R1–R11 y la lista de congelación). Es mejor una memoria honesta con límites que una incompleta. Preguntad al equipo y, después, al docente.